

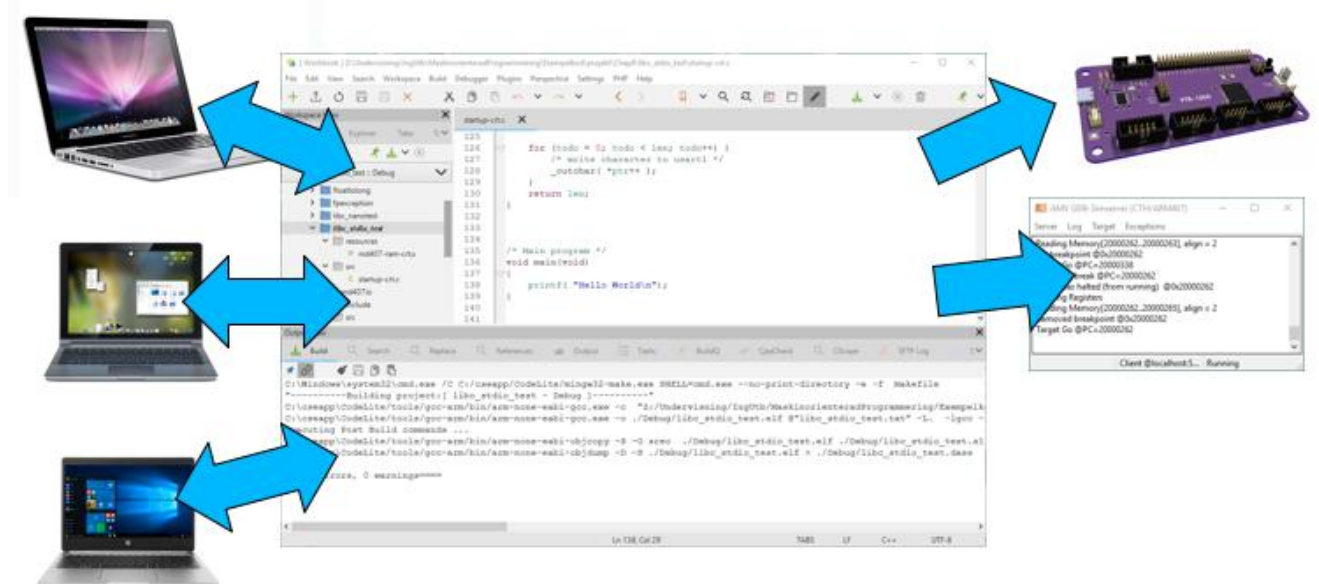
Programbibliotek

Ur innehållet:

Exekveringsmiljö

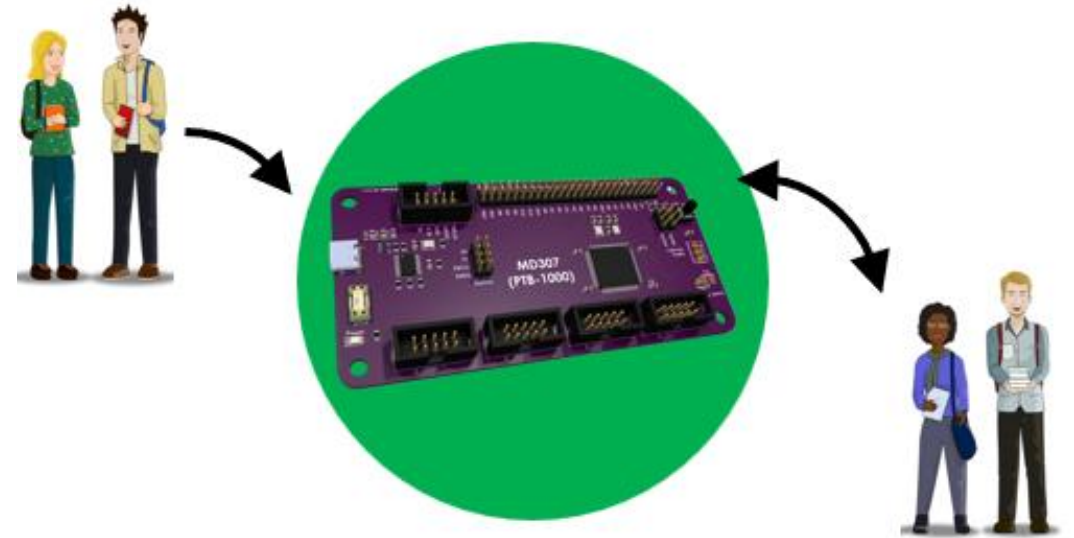
Olika typer av *programbibliotek*

- Kompilatorbibliotek
- Standard C-bibliotek
- Användarspecifika bibliotek



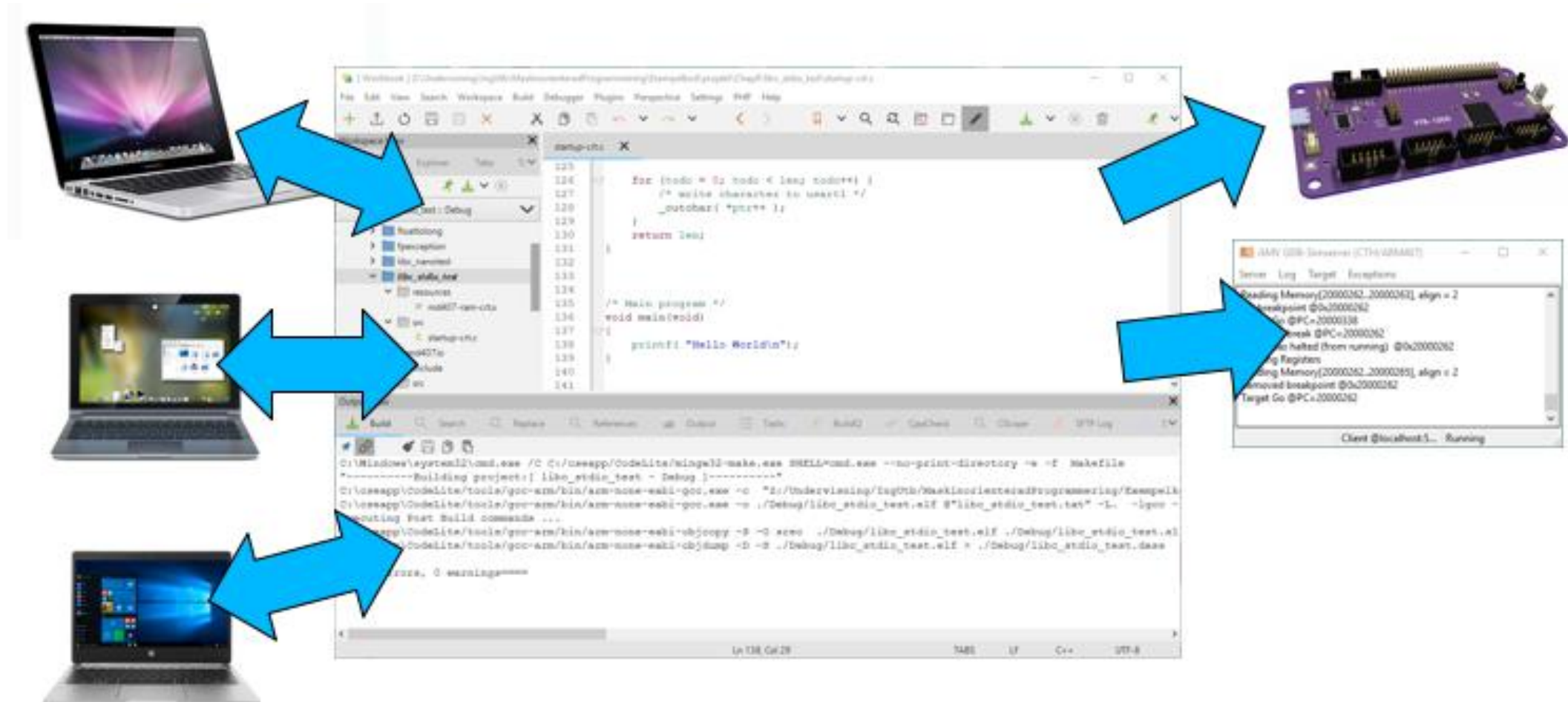
Målsättning:

Att kunna använda färdiga programbibliotek
i en icke trivial applikation för *MD307*



Värddator och måldator

Konventionell programutveckling sker på värdator *för värdator*.
Korsutveckling sker på värdator *för måldator*, i vårt fall MD307.



System-/Applikations- programmering för MD307

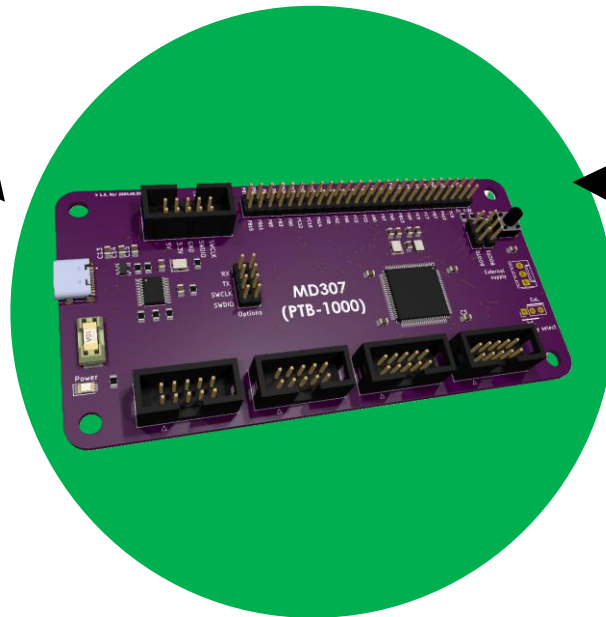
Med *programbibliotek*, avser vi att skapa grundläggande funktionalitet för användningen av vårt datorsystem. *Exempel: In-/ut-matning, filhantering, fönsterhantering etc.*

Med *systemprogramvara* menar vi oftast "basal" programvara för vår dator, ett programlager med generellt användbar natur. *Exempel: operativsystem och drivrutiner.*

Med *applikationsprogramvara*, menar vi program avsedda för mer specifik användning. *Exempel: Spel, navigation, övervakning/styrning etc.*



Systemutvecklare:
MOP studenter



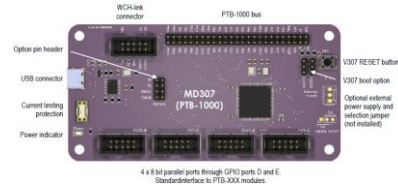
Applikationsutvecklare:
Ex: "Datorgrafik"- studenter



Måldator – specifikation

En måldator karakteriseras framför allt av:

processorarkitektur (ISA/ABI), minnesdisposition och periferienheter.



Måldator – ISA

ISA - Instruction Set Architecture

RISC-V, tillhandahåller processorer som kan hantera olika typer av instruktionsuppsättningar:

- Basuppsättningar, en av de som finns...

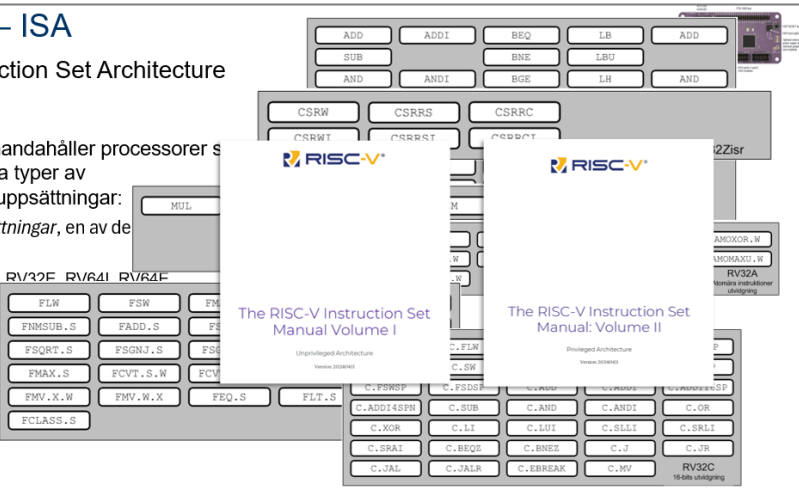
- RV32I, RV32E, RV64I, RV64E

- Åtkomst till

- RV32Z

- Utvidgningar:

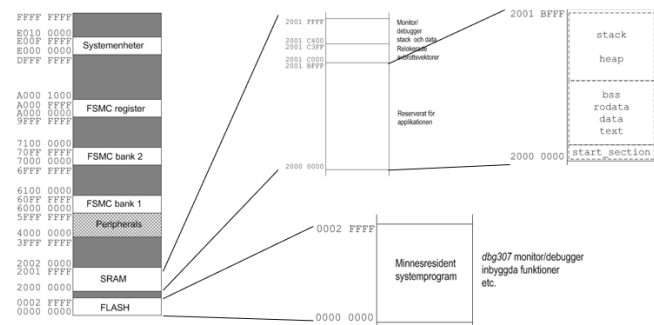
- M
- A
- F
- C
- D
- ...



Måldator – minnesdisposition

Information som måste finnas för att vi ska kunna implementera *dynamisk minneshantering*.

- Minneskonfiguration – Hur mycket minne finns det och var är det placerat i adressrummet?



Måldator – ABI

ABI - Application Binary Interface

Register	ABI Name	Description	Saver
x0	zero	Zero constant	—
x1	ra	Return address	Callee
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporaries	Callee
x8	s0 / fp	Saved / frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Fn args/return values	Callee
x12-x17	a2-a7	Fn args	Callee
x18-x27	t2-t11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Callee
f0-f7	ft0-ft7	FP temporaries	Callee
f8-9	fs0-fs1	FP saved registers	Callee
f10-f11	fa0-fa1	FP args/return values	Callee
f12-f17	fa2-fa7	FP args	Callee
f18-27	fs2-fs11	FP saved registers	Callee
f28-31	ft8-ft11	FP temporaries	Callee

RISC-V ABIs Specification
Version 1.0: Ratified

3. Sätt upp pekare till den nya aktiveringsposten, dvs. s0.

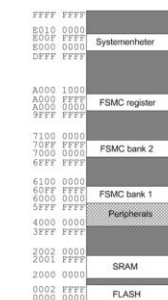
6. Frigör det reserverade stackutrymmet

Måldator – periferienheter

- Tillgängliga periferienheter –

Systemenhet

IO-enheter



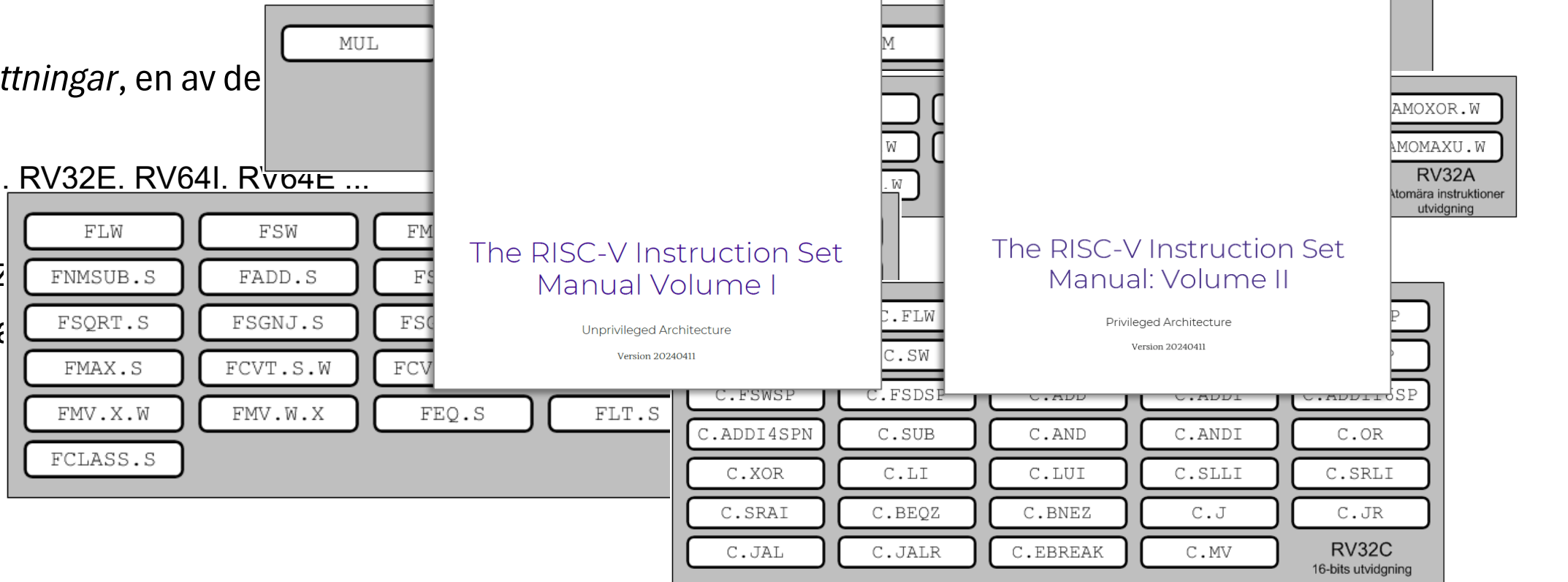
Informationen måste finnas för att vi ska kunna implementera *drivrutiner* för enheterna

Måldator – ISA

ISA - Instruction Set Architecture

RISC-V, tillhandahåller processorer som hantera olika typer av instruktionsuppsättningar:

- *Basuppsättningar*, en av de finnas...
 - RV32I. RV32E. RV64I. RV64E ...
- Åtkomst till...
 - RV32Z
- Utvidgningar...
 - M
 - A
 - F
 - C
 - D
 - ...



ABI - Application Binary Interface



Register	ABI Name	Description	Saver
x0	zero	Zero constant	—
x1	ra	Return address	Callee
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporaries	Caller
x8	s0 / fp	Saved / frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Fn args/return values	Caller
x12-x17	a2-a7	Fn args	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller
f0-7	ft0-7	FP temporaries	Caller
f8-9	fs0-1	FP saved registers	Callee
f10-11	fa0-1	FP args/return values	Caller
f12-17	fa2-7	FP args	Caller
f18-27	fs2-11	FP saved registers	Callee
f28-31	ft8-11	FP temporaries	Call

Register	ABI Namn	Användning	Sparas av:
x0	zero	alltid 0, kan inte ändras	
x1	ra	länkregister (returadress)	
x2	sp	stackpekare	
x3	gp	global pekare (basadress till globala variabler)	
x4	tp	trådpekare (basadress till aktuell tråd)	
x5-x7	t0-t2	Dessa register är avsedda som temporära. Om dom används måste dom sparas och återställas efter den anropade (callee) funktionen.	
x8	s0 (fp)	sparad pekare till aktiveringspost	
x9	s1	sparat register	
x10-x11	a0-a1	Parametrar resp. returvärdet	
x12-x17	a2-a7	Parameters	
x18-x27	s2-s11	sparade register	
x28-x31	t3-t6	temporaries	
f0-f7	ft0-ft7	floating point temporaries	
f8-f9	fs0-fs1	floating point saved registers	
f10-f11	fa0-fa1	floating point Parameters/Return values	
f12-f17	fa2-fa7	floating point Parameters	
f18-f27	fs2-fs11	floating point saved registers	
f28-f31	ft8-ft11	floating point temporaries	



RISC-V ABIs Specification

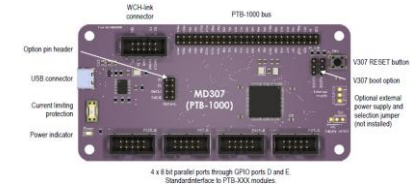
Version 1.0: Ratified

Aktiveringspost:

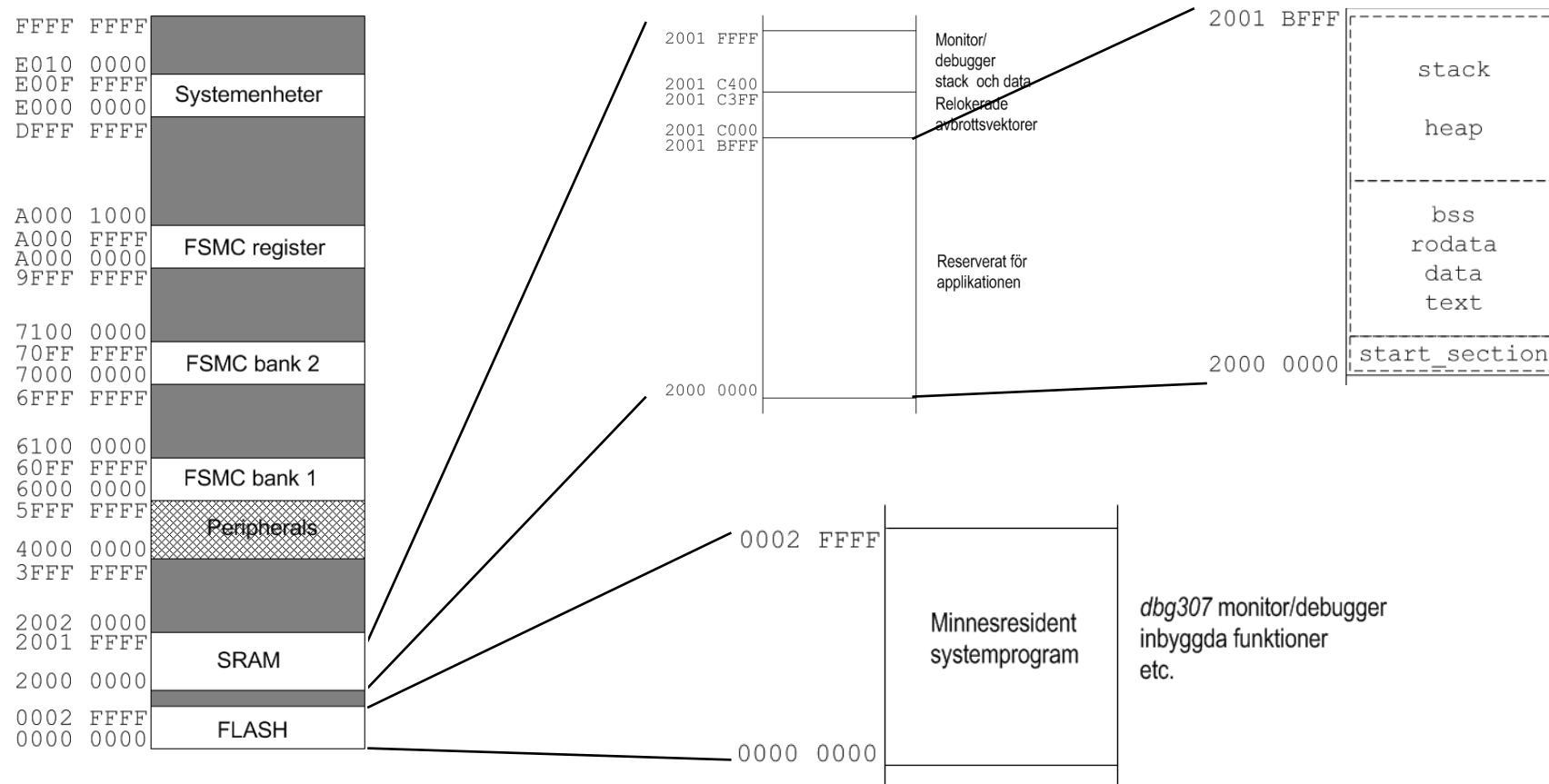
Inträde, "prolog"	
1. Reservera utrymme på stacken	5. Återställ register från stacken. Som minst register s0.
2. Spara register som används i funktionen (se tabellen ovan, "Sparas av=anropad"). Det innebär som minst register s0.	6. Frigör det reserverade stackutrymmet
3. Sätt upp pekare till den nya aktiveringsposten, dvs. s0.	

Måldator – minnesdisposition

Information som måste finnas för att vi ska kunna implementera *dynamisk minneshantering*.



- Minneskonfiguration – Hur mycket minne finns det och var är det placerat i adressrummet?



Måldator – periferienheter

- Tillgängliga periferienheter –
Systemenheter
IO-enheter

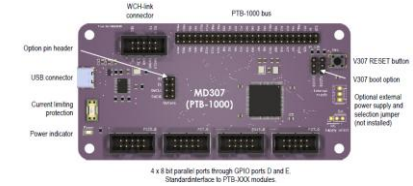
FFFF FFFF	
E010 0000	
E00F FFFF	Systemenheter
E000 0000	
DFFF FFFF	
A000 1000	
A000 FFFF	FSMC register
A000 0000	
9FFF FFFF	
7100 0000	
70FF FFFF	FSMC bank 2
7000 0000	
6FFF FFFF	
6100 0000	
60FF FFFF	FSMC bank 1
6000 0000	
5FFF FFFF	Peripherals
4000 0000	
3FFF FFFF	
2002 0000	
2001 FFFF	SRAM
2000 0000	
0002 FFFF	
0000 0000	FLASH

E000 F000
E000 E000

"SYSTICK"

"PFIC"

5005 0000 - 5005 03FF	DVP
5000 0000 - 5000 03FF	USBFS/OTG FS
4002 8000 - 4002 63FF	Ethernet
4002 3C00 - 4002 3FFF	TRNG
4002 3800 - 4002 3BFF	EXTEND
4002 3400 - 4002 37FF	USBHS
4002 3000 - 4002 33FF	CRC
4002 2000 - 4002 23FF	Flash Interface
4002 1000 - 4002 13FF	RCC
4002 0400 - 4002 07FF	DMA2
4002 0000 - 4002 03FF	DMA1
4001 8000 - 4001 83FF	SDIO
4001 5000 - 4001 53FF	TIM10
4001 4C00 - 4001 4FFF	TIM9
4001 3600 - 4001 3BFF	USART1
4001 3400 - 4001 37FF	TIM8
4001 3000 - 4001 33FF	SPI1
4001 2C00 - 4001 2FFF	TIM1
4001 2800 - 4001 2BFF	ADC2/TouchKey
4001 2400 - 4001 27FF	ADC1/TouchKey
4001 1800 - 4001 1BFF	GPIO E
4001 1400 - 4001 17FF	GPIO D
4001 1000 - 4001 13FF	GPIO C
4001 0C00 - 4001 0FFF	GPIO B
4001 0800 - 4001 0BFF	GPIO A
4001 0400 - 4001 07FF	EXTI
4001 0000 - 4001 03FF	AFIO
4000 7400 - 4000 77FF	DAC
4000 7000 - 4000 73FF	PWR
4000 6C00 - 4000 6FFF	PKP
4000 6800 - 4000 6BFF	hxCAN2
4000 6400 - 4000 67FF	hxCAN1
4000 6000 - 4000 63FF	share 512B SRAM
4000 5800 - 4000 5BFF	I2C2
4000 5400 - 4000 57FF	I2C1
4000 5000 - 4000 53FF	UART5
4000 4C00 - 4000 4FFF	UART4
4000 4800 - 4000 4BFF	USART3
4000 4400 - 4000 47FF	USART2
4000 3C00 - 4000 3FFF	SPG1/2S3
4000 3800 - 4000 3BFF	SPG1/2S2
4000 3000 - 4000 33FF	WWDG
4000 2C00 - 4000 2FFF	WWDG
4000 2800 - 4000 2BFF	RTC
4000 2000 - 4000 23FF	UART8
4000 1C00 - 4000 1FFF	UART7
4000 1800 - 4000 1BFF	UART6
4000 1400 - 4000 17FF	TIM7
4000 1000 - 4000 13FF	TIM6
4000 0C00 - 4000 0FFF	TIM5
4000 0800 - 4000 0BFF	TIM4
4000 0400 - 4000 07FF	TIM3
4000 0000 - 4000 03FF	TIM2



Informationen måste finnas för att vi ska kunna implementera *drivrutiner* för enheterna

Programbibliotek – olika typer

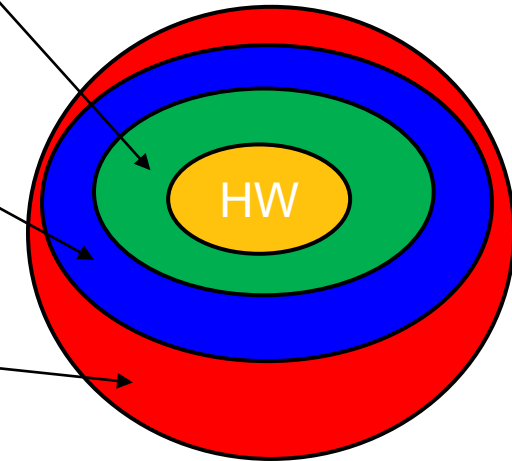
Programbibliotek kan ofta ses som "kompatibilitetslager" där en anpassning sker till någon speciell maskinvara och något specifikt operativsystem.

- **Kompilatorbibliotek** – tillhandahåller det stöd som krävs för att ett standardiserat C-program korrekt ska kunna översättas till maskinkod, oavsett processorarkitektur.
- **Standard C-bibliotek** – tillhandahåller en lång rad funktioner användbara i de flesta applikationer. Utgör också vanligtvis det viktigaste gränssnittet mot det använda operativsystemet.
- **Användarspecifika bibliotek.**

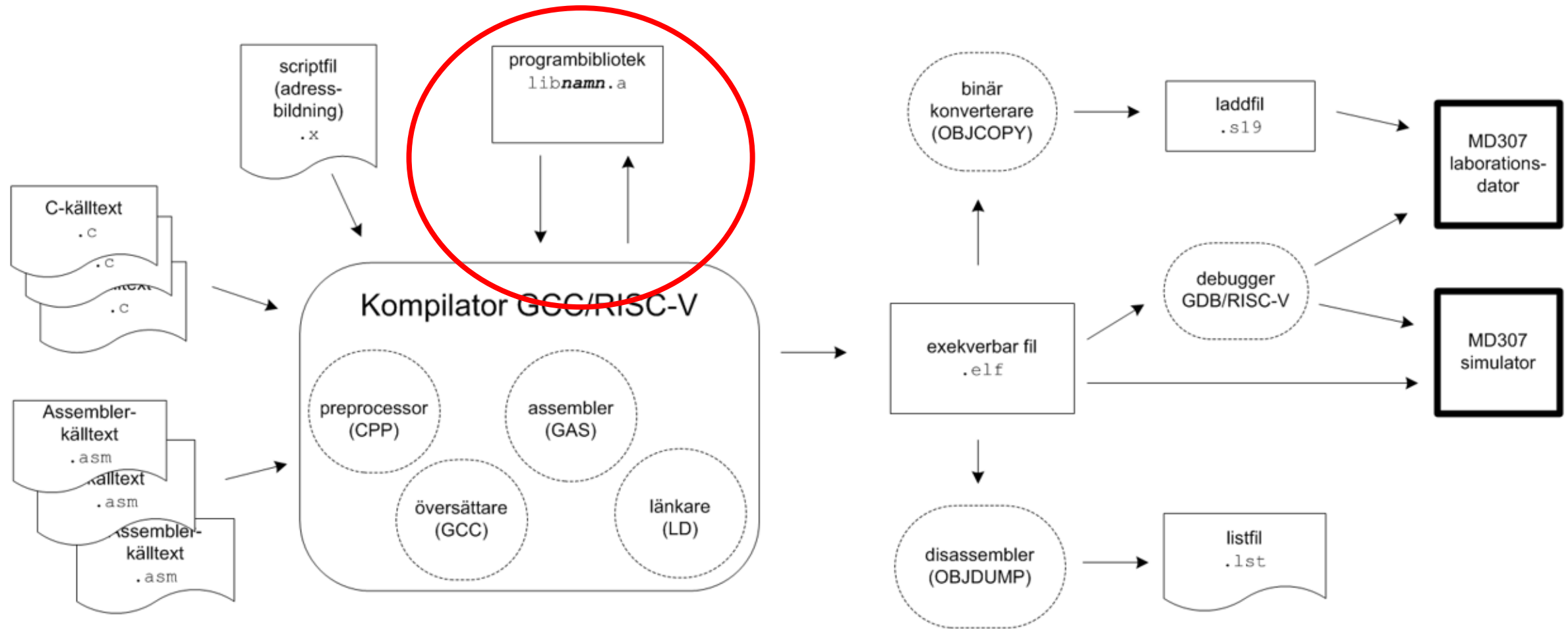
Exempel:
Flyttalsoperationer

Exempel:
malloc/free, printf...

Exempel: GUI: Fönster
och menyer

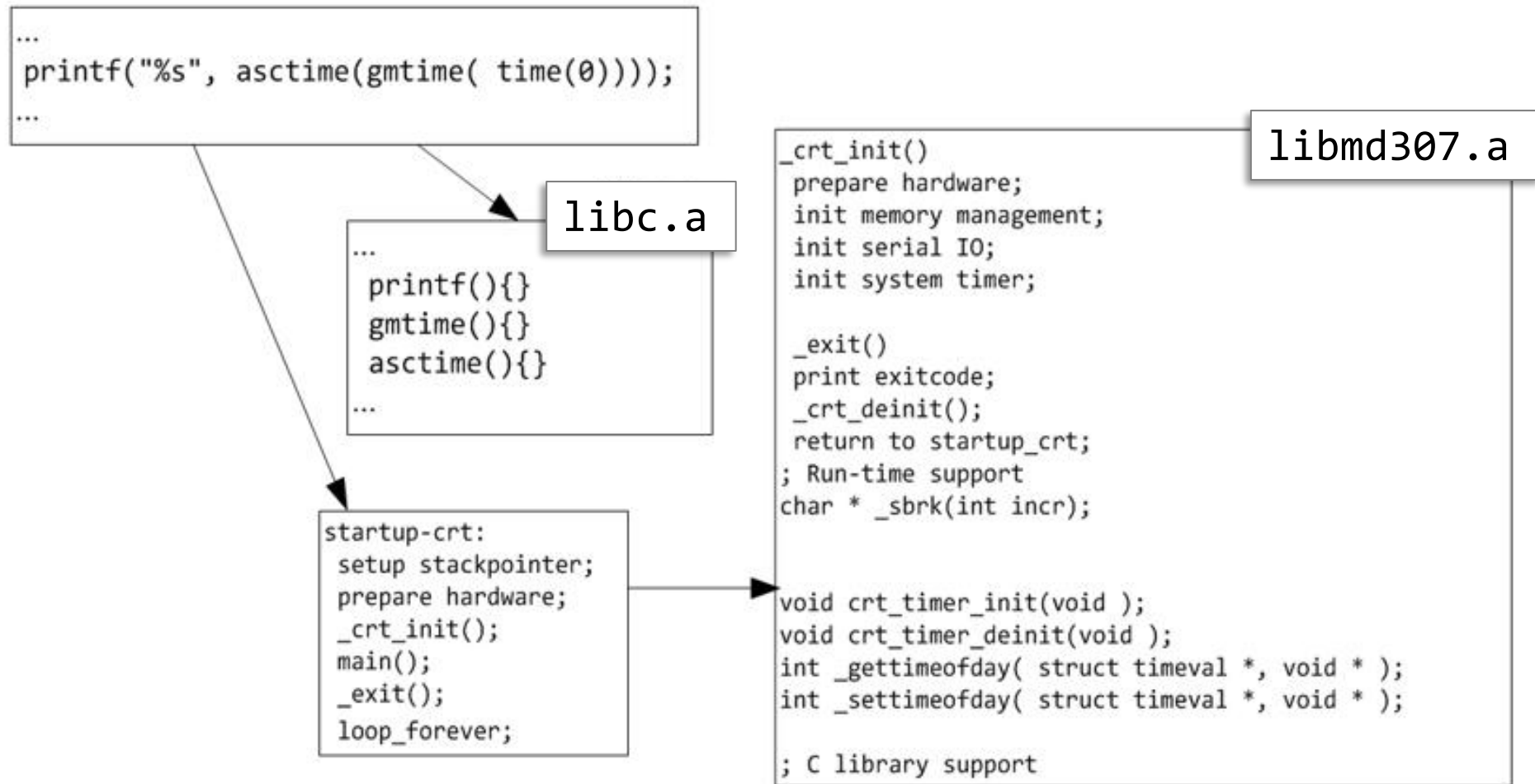


Programbibliotek - del i utvecklingsflödet



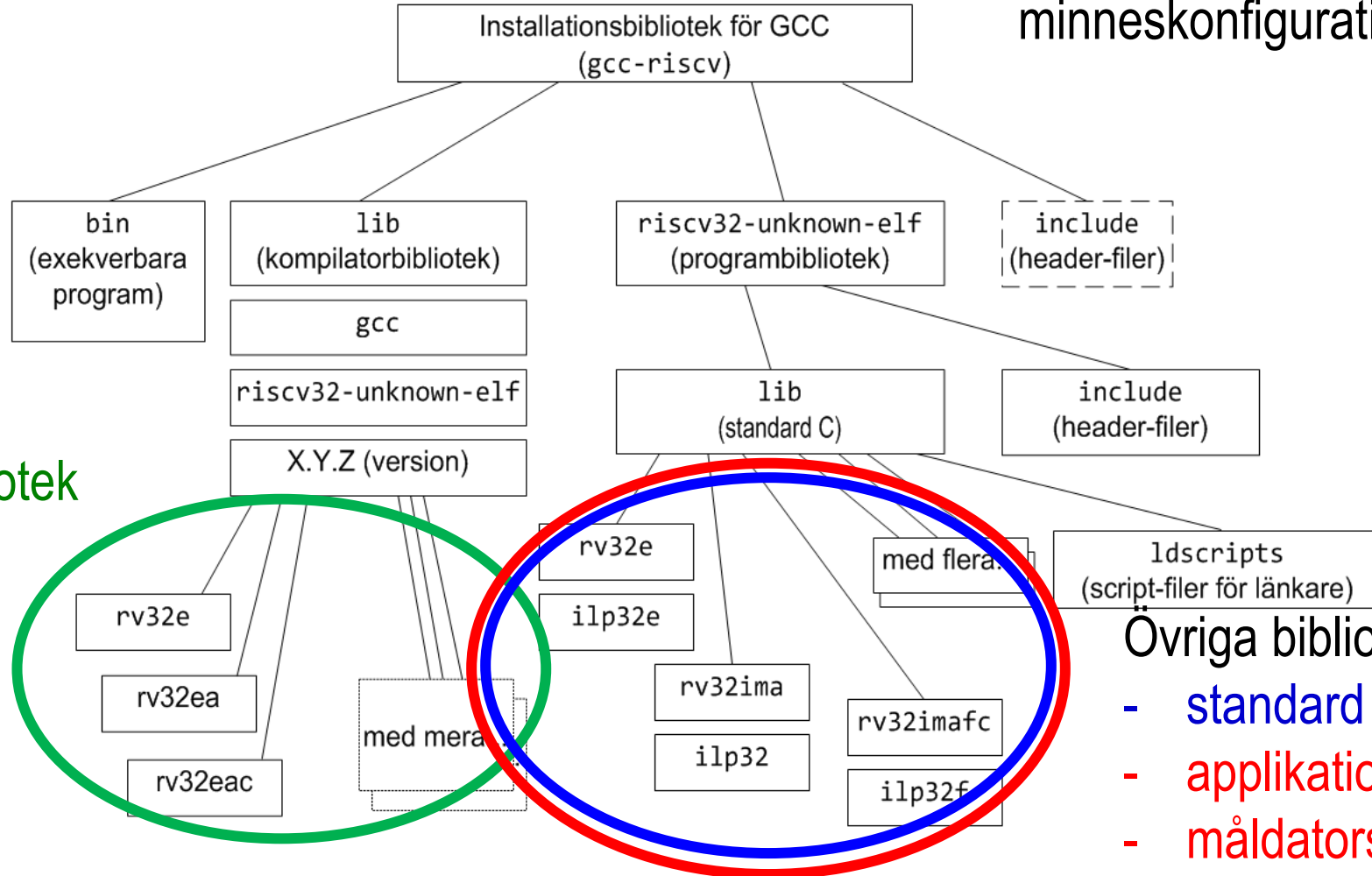
Programbibliotek - beroenden

in- och utmatning i C-biblioteket kräver "stdio" - drivrutiner för *MD307*
tidsfunktioner i C-biblioteket kräver "time" - drivrutiner för *MD307*.



Programbibliotekens organisation

Hierarkin av programbibliotek måste vara anpassad för olika processorer med olika minneskonfigurationer och periferekretsar.



Kompilatorbibliotek

Övriga bibliotek

- standard C
- applikationsspecifikt
- måldatorspecifikt

Funktionerna i kompilatorbiblioteket används av kompilatorn internt för kodgenerering, dvs. är ej publika det behövs därför ingen header-fil med deklarationer. Information om processorarkitektur måste vara konsistent vid kompilering och länkning.

Att styra kodgenereringen - GCC/RISC-V

-march=rvXXX...

RV32E	32-bitars dataväg med 16 generella register
RV32I	32-bitars dataväg med 32 generella register
RV64I	64-bitars dataväg med 32 generella register
RV128I	128-bitars dataväg med 32 generella register

A	Atomära instruktioner med CSR
C	Komprimerade instruktioner (16-bitars operationskod)
F	Enkel (32-bitars) precision flyttal
M	Multiplikation och division med heltal

-mabi=ilpXXx

ilp32	generella register används för heltal och pekare, för flyttal används stacken
ilp32f	generella register används för heltal och pekare, för flyttal används 32-bitars flyttalsregister
ilp32d	generella register används för heltal och pekare, för flyttal används 64-bitars flyttalsregister
ilp32e	generella register används för heltal och pekare. Stackpekaren kan vara rättad på 32-bitars gräns. Register x16-x31 används inte så endast register a0-a5 används för parametrar, registren s0 och s1 sparas av anropad funktion och t0,t1 och t2 är temporärregister.

För *MD307* har vi: **-march=rv32imacf -mabi=ilp32f**

Att styra kodgenereringen - *Exempel*

Heltalsmultiplikation

```
int a,b,c;  
a = b * c;
```

Om utvidgningen M finns, används maskininstruktionen **mul**.

Om utvidgningen M *inte* finns, används en funktion **__mulsi3** från i kompilatorbiblioteket i stället

```
-march=rv32im  
    lw      a1,c  
    lw      a0,b  
    mul     a0,a0,a1  
    sw      a0,a,t0
```

```
-march=rv32i  
    lw      a1,c  
    lw      a0,b  
    call    __mulsi3  
    sw      a0,a,t0
```

Flyttalsmultiplikation

```
float a,b,c;  
a = b * c;
```

Om utvidgningen F finns, genereras kod för flyttalsenheten.

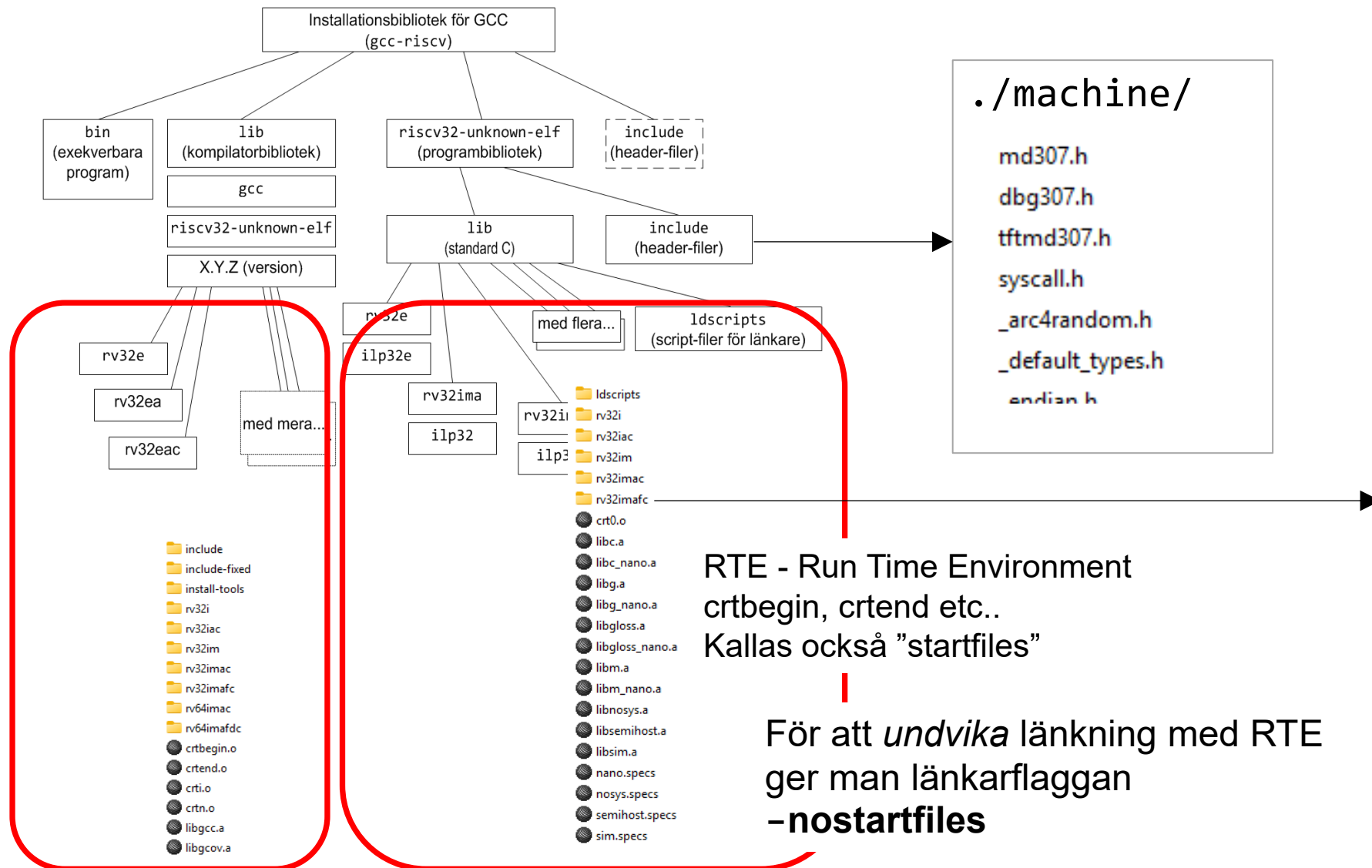
Om utvidgningen F *inte* finns, används en funktion **__mulsf3** från i kompilatorbiblioteket i stället

```
-march=rv32imfac  
    flw     fa1,c  
    flw     fa0,b  
    fmul.s  fa0,fa0,fa1  
    fsw     a0,a,t0
```

```
-march=rv32im  
    lw      a1,c  
    lw      a0,b  
    call    __mulsf3  
    sw      a0,a,t0
```

Kompilatorbiblioteket tillhandahåller funktioner för alla operationer (med alla datatyper) som kan utföras i programspråket C.

Programbiblioteken - varianter i en *md307*- miljö



RTE - Run Time Environment
crtbegin, crtend etc..
Kallas också "startfiles"

För att *undvika* länkning med RTE
ger man länkarflaggan
-nostartfiles

./ilp32f/
libdbg307.a
libdbg307-cold.a
libdbg307d.a
libdbg307-sim.a
libmd307.a
libmd307-cold.a
libmd307d.a
libmd307-sim.a
libtftmd307.a
libtftmd307-cold.a
libtftmd307d.a
libtftmd307-sim.a
crt0.o
libc_nano.a
libg_nano.a
libgloss_nano.a
libm_nano.a
libgloss.a
libnosys.a
libsemihost.a
libsim.a
nano.specs
nosys.specs
semihost.specs
sim.specs
libc.a
libg.a
libm.a

Demonstration: Användning av biblioteket libdbg307

Speciella programbibliotek MD307

```
void dbg_restart(void);
void dbg_clockinit(void);
void dbg_portinit(void);
void dbg_outchar(unsigned char);
unsigned char dbg_tstchar(void);
int dbg_tft_init(int option);
void dbg_tft_lcd_ellipse(int,int,int,int,int,int);
void dbg_tft_lcd_rect(int,int,int,int,int,int);
void dbg_tft_lcd_line(int,int,int,int,int);
void dbg_tft_lcd_pixe
void dbg_delay_ms(uns
int dbg_tft_tp_scan(i
int dbg_tft_tp_getpos
```

Headerfil:

```
#include <machine/dbg307.h>
```

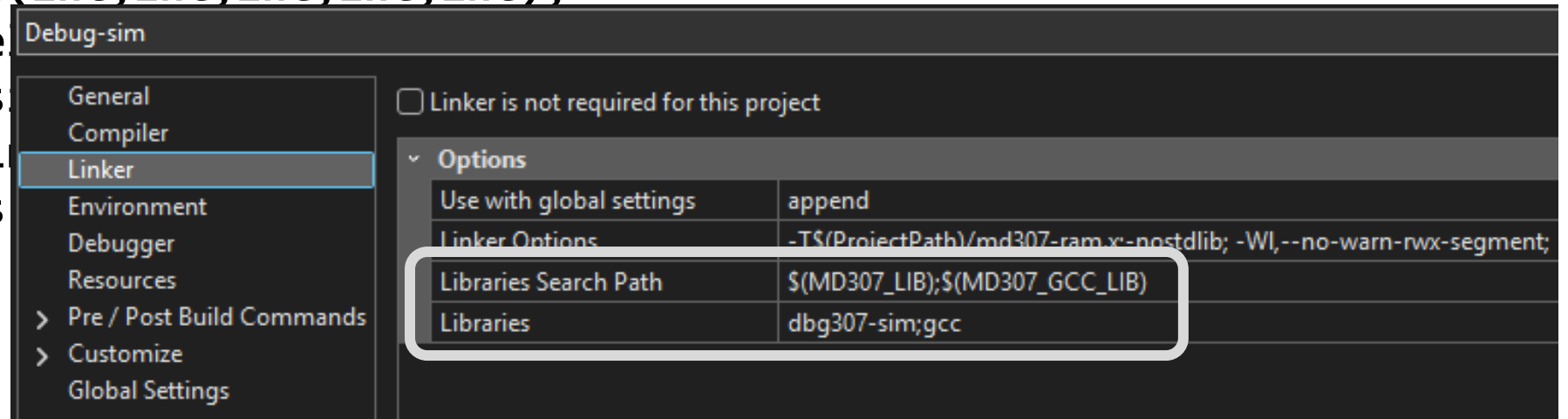
Programbibliotek

libdbg307d.a

libdbg307-cold.a

libdbg307-sim.a

libdbg307.a



Exempel: Användning av programbiblioteket

Headerfil:
`#include <machine/dbg307.h>`

Programbibliotek
`libdbg307d.a`
`libdbg307-cold.a`
`libdbg307-sim.a`
`libdbg307.a`

```
void dbg_restart(void);  
void dbg_clockinit(void);  
void dbg_portinit(void);  
void dbg_outchar(unsigned char);  
unsigned char dbg_tstchar(void);  
int dbg_tft_init(int option);  
void dbg_tft_lcd_ellipse(int,int,int,int,int,int);  
void dbg_tft_lcd_rect(int,int,int,int,int,int);  
void dbg_tft_lcd_line(int,int,int,int,int);  
void dbg_tft_lcd_pixel(int,int,int);  
void dbg_delay_ms(unsigned int);  
int dbg_tft_tp_scan(int);  
int dbg_tft_tp_getpos(int *,int *);
```

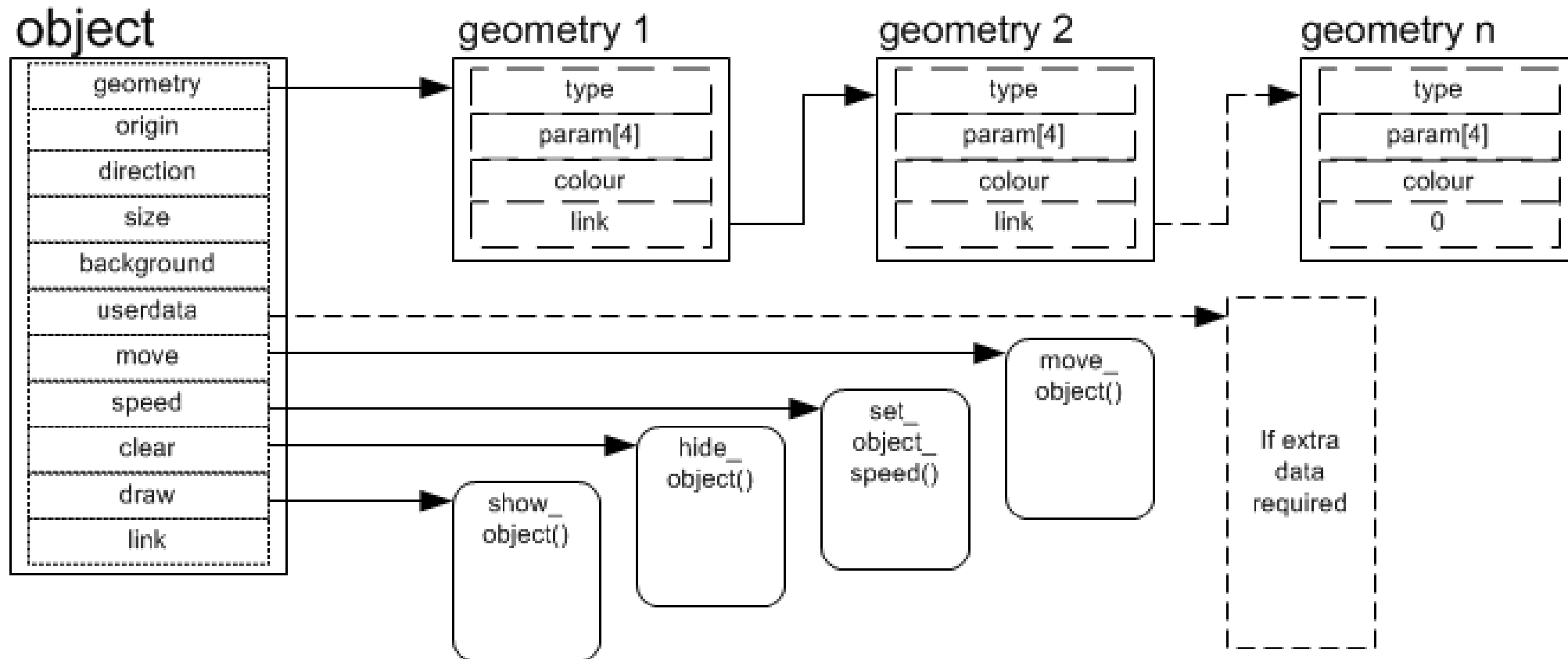


```
#include <machine/dbg307.h>  
  
void main(void)  
{  
    if( ! dbg_tft_init(TFT_INIT_ILI9488) )  
        return;  
  
    for( int i = 0; i < sizeof(lines)/sizeof( LINE ); i++ )  
    {  
        dbg_tft_lcd_line(lines[i].x1, lines[i].y1, lines[i].x2, lines[i].y2, TFT_RED);  
    }  
    dbg_delay_ms( 3000 );  
    dbg_tft_lcd_rect(0, 0, 479, 319, TFT_CYAN, 1);  
    for( int i = 0; i < sizeof(lines)/sizeof( LINE ); i++ )  
    {  
        dbg_tft_lcd_line(lines[i].x1, lines[i].y1, lines[i].x2, lines[i].y2, TFT_RED);  
    }  
}
```

Mer avancerade objekt

Vi sätter samman objekt av de grundläggande geometrierna.

```
void draw_object( POBJECT o )
{
    PGEOMETRY g = o->geometry;
    while( g )
    {
        draw_geometry( g, o->x, o->y, g->colour );
        g = g->link;
    }
}
```

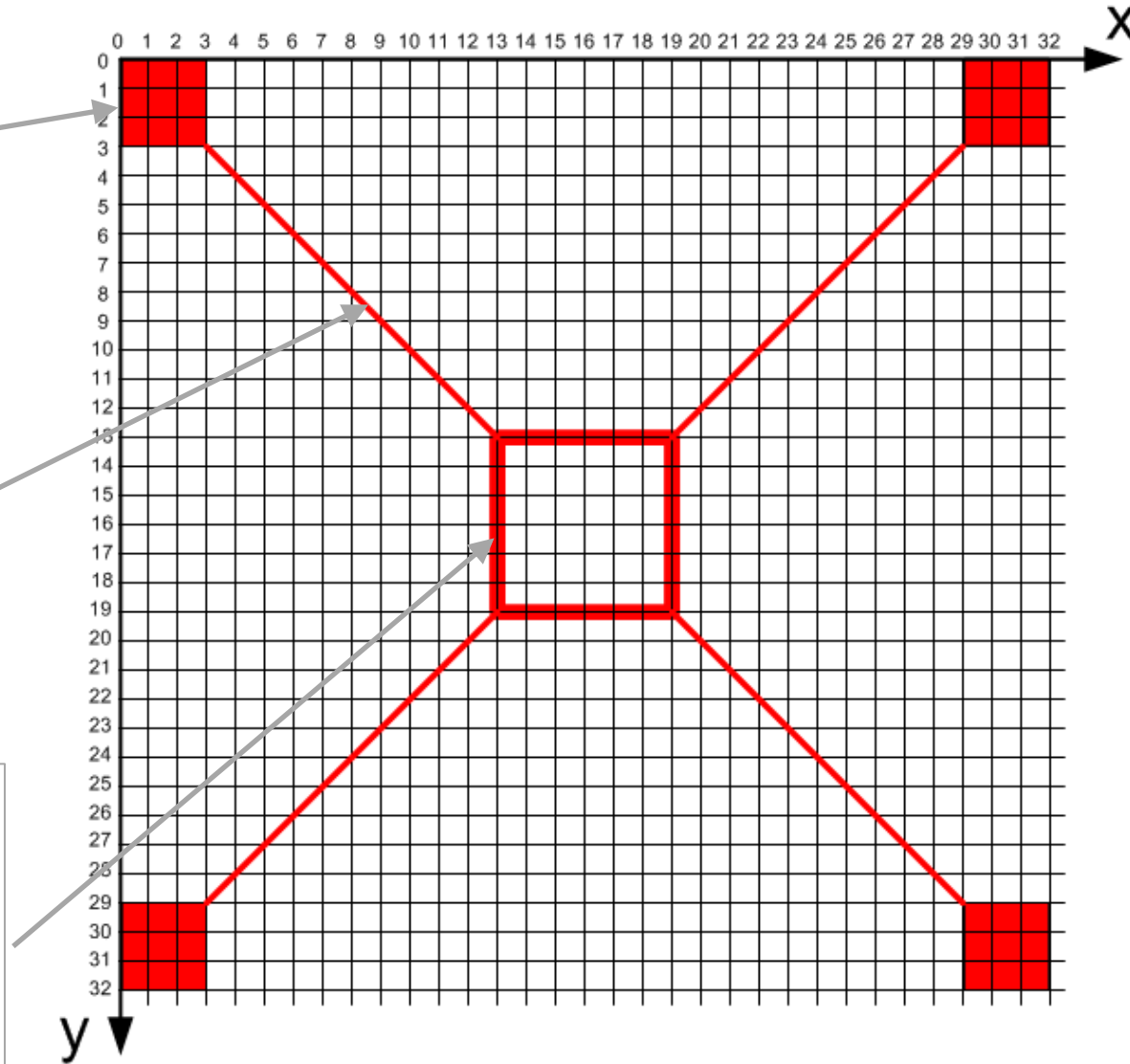


Exempel, object med flera geometrier (1) Objekt "drönare"

```
static GEOMETRY rect1 =  
{  
    GEO_FILLED_RECT,  
    0,0,  
    3,3, TFT_RED,  
    &rect2  
};
```

```
static GEOMETRY line1 =  
{  
    GEO_LINE,  
    3,3,  
    13,13,  
    TFT_RED,  
    &line2  
};
```

```
static GEOMETRY rect0 =  
{  
    GEO_RECT,  
    13,13,  
    6,6,  
    TFT_RED,  
    &rect1  
};
```



```
static OBJECT drone =  
{  
    &rect0,  
    10,10,  
    5,2,  
    32,32,  
    TFT_WHITE,  
    draw_bouncer,  
    clear_bouncer,  
    move_object,  
    set_object_speed  
};
```


Exempel, object med flera geometrier (2)

En liten "drönare" som kan styras från "KeyPad".



Kameraupptagning



Simulator

"clib" - MD307 implementering av vitala Standard C-funktioner

<code><assert.h></code>	Definierar makro som kan användas för att upptäcka logiska fel och andra felaktigheter i "debug"-versioner av program.
<code><ctype.h></code>	Deklarerar funktioner för att klassificera och omvandla tecken.
<code><errno.h></code>	För att kontrollera felkoder från biblioteksfunktionerna.
<code><float.h></code>	Definierar olika konstanter som definierar implementationsspecifika detaljer i flyttalshanteringen.
<code><limits.h></code>	Definierar olika konstanter som definierar implementationsspecifika detaljer i heltalshanteringen.
<code><locale.h></code>	Deklarerar funktioner för lokala anpassningar.
<code><math.h></code>	Deklarerar vanliga matematiska funktioner..
<code><setjmp.h></code>	Deklaration för speciella programflödesändringar.
<code><signal.h></code>	Deklarerar funktioner för hantering av "signaler".
<code><stdarg.h></code>	För variabelt antal argument till en funktion.
<code><stddef.h></code>	Definierar diverse olika typer och macron.
<code><stdio.h></code>	Deklarerar in- och utmatningsfunktioner.
<code><stdlib.h></code>	Deklarerar funktioner för numeriska konverteringar, slumpatsgenerering och minneshantering.
<code><string.h></code>	Deklarerar funktioner för stränghantering.
<code><time.h></code>	Deklarerar funktioner för datum och tid.

Vi gör ingen fullständig implementering av alla delar, exempelvis implementeras enbart terminal-IO för *konsoll* i `stdio`.

"clib" - dynamisk minneshantering

<assert.h>	Definierar makro som kan användas för att upptäcka logiska fel och andra felaktigheter i "debug"-versioner av program.
<ctype.h>	Deklarerar funktioner för att klassificera och omvandla tecken.
<errno.h>	För att kontrollera felkoder från biblioteksfunktionerna.
<float.h>	Definierar olika konstanter som definierar implementationsspecifika detaljer i flyttalshantering.
<limits.h>	Definierar olika konstanter som definierar implementationsspecifika detaljer i heltalshantering.
<locale.h>	Deklarerar funktioner för lokala anpassningar.
<math.h>	Deklarerar vanliga matematiska funktioner..
<setjmp.h>	Deklaration för speciella programflödesändringar.
<signal.h>	Deklarerar funktioner för hantering av "signaler".
<stdarg.h>	För variabelt antal argument till en funktion.
<stddef.h>	Definierar diverse olika typer och macron.
<stdio.h>	Deklarerar in- och utmatningsfunktioner.
<stdlib.h>	Deklarerar funktioner för numeriska konverteringar, slumpalgsgenerering och minneshantering.
<string.h>	Deklarerar funktioner för stränghantering.
<time.h>	Deklarerar funktioner för datum och tid.

Funktioner för dynamisk minneshantering:

```
void *malloc(size_t size);  
void *calloc(size_t items, size_t size);  
void *realloc(void * ptr, size_t size);  
void free(void *ptr);
```

EXEMPEL 6.14 ANVÄNDNING AV MALLOC/FREE

Allokera 100 bytes minne.

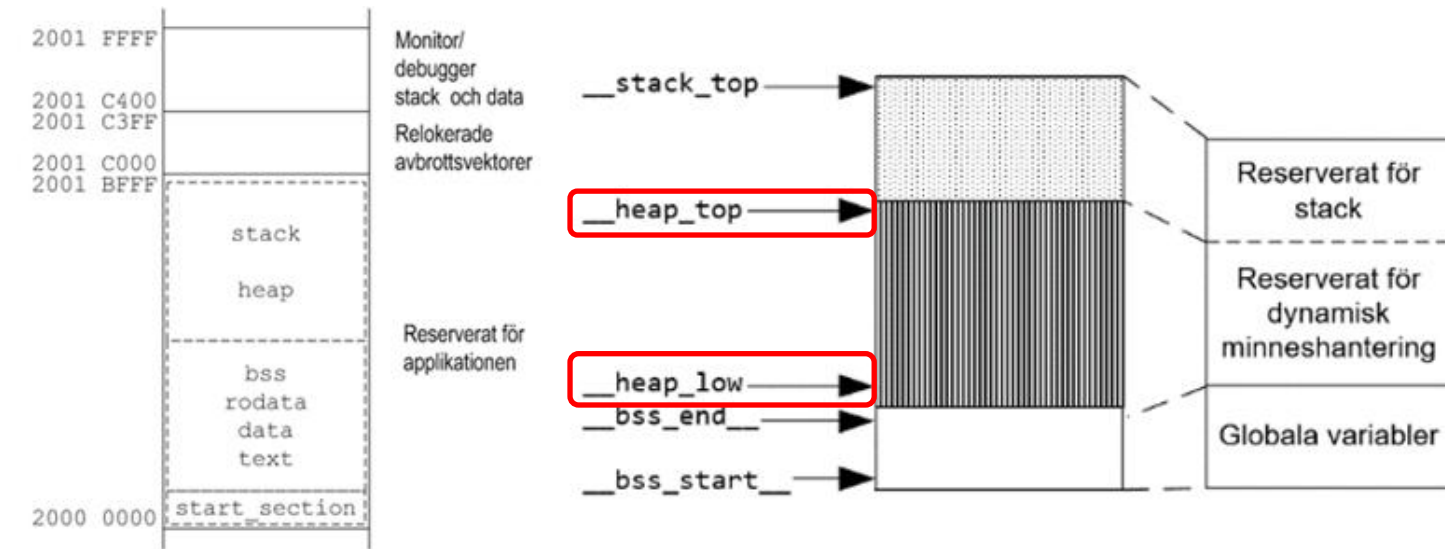
```
char *dbuf = (char *) malloc( 100 );  
if( dbuf == (char *) 0 )  
    .... /* error */
```

Observera hur returvärdet från **malloc** ska typkonverteras till den pekartyp man avser.

Frigör minne tidigare returnerat av malloc.

```
free(dbuf);
```

"clib" - måldatorberoende implementering minneshantering



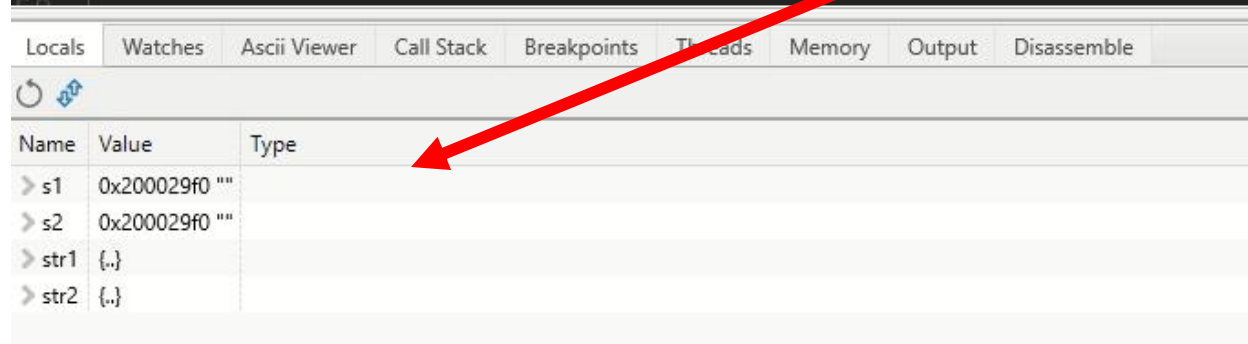
FIGUR 6.3 MINNESANVÄNDNING I MD307

```
extern char __heap_low;
extern char __heap_top;
static char *heap_end;
char * _sbrk(int incr)
{
    char *prev_heap_end;
    if (heap_end == 0) {
        heap_end = &__heap_low;
    }
    prev_heap_end = heap_end;
    if (heap_end+incr > &__heap_top){
        return (char *)-1;
    }
    heap_end += incr;
    return (char *) prev_heap_end;
}
```

Funktionen `_sbrk` anropas från `malloc` då nytt minne ska allokeras.
Därefter hanteras minnesblocken av `malloc/free`.

"clib" – *Exempel:* malloc, realloc och free

```
41
42 #include <string.h>
43 #include <stdlib.h>
44
45 void main(void)
46 {
47     char str1[]="Beginning of string";
48     char str2[]="Continuation of string";
49     char *s1,*s2;
50     s1 = malloc( strlen(str1)+1 );
51     if( s1 == 0 ) exit(-1);
52     strcpy( s1, str1 );
53     s2 = realloc( s1, strlen(str1)+strlen(str2)+1 );
54     if( s2 == 0 ) exit(-1);
55     strcat( s2,str2 );
56     free( s2 );
57 }
```



Name	Value	Type
> s1	0x200029f0 ""	
> s2	0x200029f0 ""	
> str1	{..}	
> str2	{..}	

Observera hur variablerna s1 och s2 påverkas av varje exekveringssteg i programmet.

"clib" - måldatorberoende implementering "file IO"

Standard C-biblioteket för generell filhantering är mycket omfattande,
de maskinberoende delarna kan för vår del dock begränsas till en handfull funktioner enligt:

<assert.h>	Definierar makro som kan användas för att upptäcka logiska fel och andra felaktigheter i "debug"-versioner av program.
<ctype.h>	Deklarerar funktioner för att klassificera och omvandla tecken.
<errno.h>	För att kontrollera felkoder från biblioteksfunktionerna.
<float.h>	Definierar olika konstanter som definierar implementationsspecifika detaljer i flyttalshanteringen.
<limits.h>	Definierar olika konstanter som definierar implementationsspecifika detaljer i heltalshanteringen.
<locale.h>	Deklarerar funktioner för lokala anpassningar.
<math.h>	Deklarerar vanliga matematiska funktioner..
<setjmp.h>	Deklaration för speciella programflödesändringar.
<signal.h>	Deklarerar funktioner för hantering av "signaler".
<stdarg.h>	För variabelt antal argument till en funktion.
<stddef.h>	Definierar diverse olika typer och macron.
<stdio.h>	Deklarerar in- och utmatningsfunktioner.
<stdlib.h>	Deklarerar funktioner för numeriska konverteringar, slumpvalsgenerering och minneshantering.
<string.h>	Deklarerar funktioner för stränghantering.
<time.h>	Deklarerar funktioner för datum och tid.

Systemfunktioner för filhantering

```
int _open(const char *name, int flags, int mode);
int _close(int file);
int _lseek(int file, int ptr, int dir);
int _fstat(int file, struct stat *st);
int _isatty(int file);
int _read(int file, char *ptr, int len);
int _write(int file, char *ptr, int len);
```

Parametern `file` är en unik filidentifierare som returneras av systemanropet `_open`.

Denna sparas och används för alla andra operationer på filen.

Anm. I C-biblioteket finns bara funktionerna `open`, `close` etc. (utan `_`) synliga för applikationsprogrammeraren

"clib" - måldatorberoende implementering "terminal IO"

filidentifierare 0,1 och 2 är fördefinierade och representerar systemets kommunikation med en *konsoll*, dvs. en terminal som är ASCII-tecken orienterad och primär kommunikationskanal med datorn, (jämför kapitel 5 om seriekommunikation):

```
(i <machine/md307.h>
#define STDIN_FILENO  0
#define STDOUT_FILENO 1
#define STDERR_FILENO 2
```

fil för att läsa från systemets konsoll
fil för att skriva till systemets konsoll
fil för att skriva felmeddelanden till systemets konsoll

Exempel:

```
fprintf( stdout, "Message: ....");
fprintf( stderr, "Error: ....");
fscanf(  stdin, "....",...);
```

motsvarar STDOUT_FILENO
motsvarar STDERR_FILENO
motsvarar STDIN_FILENO

"clib" - MD307 implementering

"terminal IO" (1)

```
int _open(const char *name, int flags, int mode);  
int _close(int file);  
int _lseek(int file, int ptr, int dir);  
int _fstat(int file, struct stat *st);  
int _isatty(int file);  
int _read(int file, char *ptr, int len);  
int _write(int file, char *ptr, int len);
```

_open, öppna en fil för läsning och/eller skrivning.

Vi har inget filsystem för blockorienterade enheter, våra enheter (0,1,2) öppnas av runtime-systemet. Det räcker därför att vår implementering returnerar en felkod om _open anropas.

```
int _open(const char *name, int flags, int mode) { return -1; }
```

För *övriga* funktionsanrop måste vi då kontrollera att enbart de fördefinierade parametrarna (0,1,2) kan förekomma. Detta begränsar oss samtidigt till "terminal IO".

Samtidigt måste en bestämd USART anges eftersom vi bara kan ha en konsoll. För *MD307* används USART1 som seriekommunikationskrets för konsollen.

"clib" - MD307 implementering

"terminal IO" (2)

```
int _open(const char *name, int flags, int mode);  
int _close(int file);  
int _lseek(int file, int ptr, int dir);  
int _fstat(int file, struct stat *st);  
int _isatty(int file);  
int _read(int file, char *ptr, int len);  
int _write(int file, char *ptr, int len);
```

_close, stäng en fil som är öppen för läsning och/eller skrivning.
Våra enheter stängs av runtime-systemet. Vi returnerar därför felkod.

```
int _close(int file) { return -1; }
```

_lseek, sök till position i en fil som är öppen för läsning och/eller skrivning.
Måste vara en blockorienterad fil, ej tillämpligt för teckenorienterade enheter.
Vi returnerar därför felkod.

```
int _lseek(int file, int ptr, int dir) { return 0; }
```

"clib" - MD307 implementering

"terminal IO" (3)

```
int _open(const char *name, int flags, int mode);  
int _close(int file);  
int _lseek(int file, int ptr, int dir);  
int _fstat(int file, struct stat *st);  
int _isatty(int file);  
int _read(int file, char *ptr, int len);  
int _write(int file, char *ptr, int len);
```

_fstat, returnerar information om en öppen fil.

```
#include <sys/stat.h>  
int _fstat(int file, struct stat *st) {  
    if( file <= STDERR_FILENO ){  
        st->st_mode = S_IFCHR;  
        return 0;  
    }  
    return -1;  
}
```

struct stat och
S_IFCHR deklaras i
sys/stat.h

S_IFCHR, anger att detta är en teckenorienterad fil, andra exempel är:

S_IFDIR, filen är ett bibliotek.

S_IFBLK, filen är block-orienterad, (typiskt på disk) osv...

Vår implementering stödjer bara teckenorienterade enheter.

"clib" - MD307 implementering

"terminal IO" (4)

```
int _open(const char *name, int flags, int mode);
int _close(int file);
int _lseek(int file, int ptr, int dir);
int _fstat(int file, struct stat *st);
int _isatty(int file);
int _read(int file, char *ptr, int len);
int _write(int file, char *ptr, int len);
```

_isatty ska returnera 1 om filen är av terminaltyp, dvs. teckenorienterad enhet. `stdin`, `stdout` och `stderr`, kan riktas även mot blockorienterade enheter, vilket vi inte hanterar. För anrop med parameter utanför vår konsoll returneras därför felkod.

```
int _isatty(int file) {
    if( file <= STDERR_FILENO )
        return 1;
    return 0;
}
```

"clib" - MD307 implementering

"terminal IO" (5)

```
int _open(const char *name, int flags, int mode);
int _close(int file);
int _lseek(int file, int ptr, int dir);
int _fstat(int file, struct stat *st);
int _isatty(int file);
int _read(int file, char *ptr, int len);
int _write(int file, char *ptr, int len);
```

_read, läs **len** tecken till strängen **ptr** från en öppen fil (enhet).
Anropar enhetens läs-funktion om en sådan finns.

```
int _read(int file, char *ptr, int len){
    char c;
    int items = 0;
    if( file != STDIN_FILENO)
        return 0;
    while( 1 ){
        c = usart_tst(USART1);
        if( c != 0 ){
            if( c == '\r' ) c = '\n';
            *ptr++ = c;
            items++;
        }
        if( len == items)
            return items;
    }
}
```


"clib" - MD307 implementering

"terminal IO" (6)

```
int _open(const char *name, int flags, int mode);
int _close(int file);
int _lseek(int file, int ptr, int dir);
int _fstat(int file, struct stat *st);
int _isatty(int file);
int _read(int file, char *ptr, int len);
int _write(int file, char *ptr, int len);
```

_write, skriv **len** tecken från strängen **ptr** till en öppen fil (enhet).

Funktionen ska rikta operationen till enhetens *write-rutin* om en sådan finns.

I vårt fall tillåter vi enbart skrivningar till konsollen.

```
int _write(int file, char *ptr, int len){
    int rval = len;
    if( (file != STDOUT_FILENO) && (file != STDERR_FILENO) )
        return 0;
    while (len-->0)
        usart_out( USART1, *ptr++);
    return rval;
}
```

"clib" – *Exempel:* fscanf och fprintf

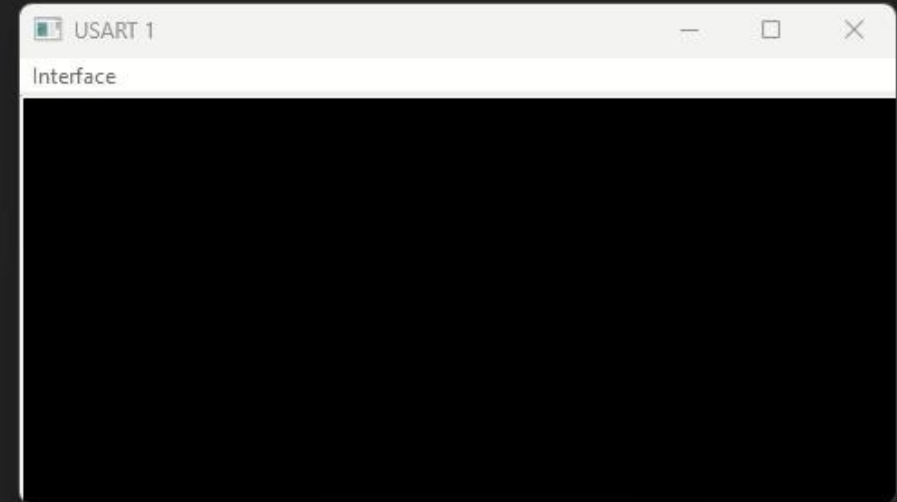
fscanf(stdin, "%s", buffer)

läs textsträng från konsollen till
strängvariabeln buffer. Avsluta vid <\n>.

fprintf(stdout, "%s\n", buffer)

Skriv innehållet i buffer som en textsträng
åtföljt av <\n> till konsollen.

```
39 #endif
40 }
41
42 #include <stdio.h>
43 int main(void)
44 {
45     char buffer[256];
46     while(1)
47     {
48         fscanf(stdin, "%s", buffer );
49         fprintf(stdout, "%s\n", buffer);
50     }
51 }
52
53
```



"clib" - MD307 tidshantering

`current_time` anger antalet 1 ms perioder sedan
"UNIX epoch" (1 Januari 1970)
Vår starttid är den **1 januari 2026**

/* Det finns en trevlig konverterare här:
<https://www.epochconverter.com>

```
int _gettimeofday( struct timeval *tv, void *tzvp ){
    if( tv == 0 )
        return -1;
    tv->tv_sec = current_time / 1000;
    tv->tv_usec = ( current_time % 1000 ) * 1000;
    return 0;
}

int _settimeofday( struct timeval *tv, void *tzvp ){
    if( tv == 0 )
        return -1;
    current_time = 1000 * tv->tv_sec ;
    current_time += tv->tv_usec;
    return 0;
}
```

```
static uint64_t current_time = 1767225600000;
__attribute__((interrupt ))
void crt_timer_interrupt( void ){
    CRT_TIM->intfr &= ~TIM_UIF;
    current_time++;
}
```

```
void crt_timer_init(){
    RCC->apb1cenr |= 0x10;
    RCC->apb1prstr |= 0x10;
    RCC->apb1prstr &= ~0x10;
    CRT_TIM->ctlr1 &= ~TIM_CEN;
    *IRQVEC(CRT_TIM_IRQ)= crt_timer_interrupt;
    *PFIC_IENR(CRT_TIM_IRQ) |=
        PFIC_IRQ_BPOS( CRT_TIM_IRQ );
    CRT_TIM->psc= 143;
    CRT_TIM->atrlr = 999;
    CRT_TIM->dmaintenr |= TIM_UIE;
    CRT_TIM->ctlr1 |= TIM_CEN;
}

void crt_timer_deinit() {
    CRT_TIM->ctlr1 &= ~TIM_CEN;
    RCC->apb1cenr &= ~0x10;
    *PFIC_IENR(CRT_TIM_IRQ) &=
        ~(PFIC_IRQ_BPOS( CRT_TIM_IRQ ));
}
```

Dessa funktioner beskrivs i
lärobokens kapitel 4

"clib" – *Exempel: MD307* tidshantering

seconds = time(NULL)

Returnera aktuell tid som time_t

tp = gmtime(& seconds)

Översätt time_t till struct gm
(greenwich meridian time)

s = asctime(tp)

Översätt struct gm till textsträng

```
# include <stdio.h>
# include <time.h>
# include <sys/time.h>

int main(void)
{
    time_t seconds, save;
    struct tm *tp;
    char *s;
    printf("Startar\n");
    save = time(NULL);
    tp = gmtime(&save);
    s = asctime(tp);
    printf( "%s", s);
    save = time(NULL);
    while( 1 )
    {
        seconds = time(NULL);
        if( seconds >= (save+1) ){
            tp = gmtime(&seconds);
            s = asctime(tp);
            printf( "%s", s);
            save = seconds;
        }
    }
}
```

USART 1

Interface

Laboration 4

Förutsättningar:

Arbeta i grupper som ni själva sätter samman. En grupp består av max 6 studenter. Vid examination ska varje gruppmedlem vara beredd att besvara frågor kring gruppens arbete (resultat). Gruppen enas om och lägger fram sitt spelförslag.

Vad krävs?

Spelförslaget är helt fritt men måste kunna realiseras med laborationssystemet *MD307*. Använd färdiga och/eller egna programbibliotek. Redovisningen består av exekverbar fil och en underhållande video där spelet demonstreras.

Vad belönas?

Väl genomfört spelförslag samt originalitet och innovation kring implementeringen.

Hur bedöms?

Spelets originalitet, interaktionsvärde och realtidsegenskaper bedöms och betygssätt enligt betygskala U,3,4,5

Inspiration:
Wikipedia
www.coolaspel.se